Penetration Testing Sandbox Lab

Created by Kelvin Nduhiu ….
Instagram @thebrawtalkdigital

Created a mock website called MockVulnerable E-Shop with a single-file application (`app.py`) featuring an e-commerce mock setup with **registration, login, a customer dashboard, and an admin dashboard**
. It contains classic security vulnerabilities (OWASP Top 10) like **SQL Injection (SQLi), Cross-Site Scripting (XSS), Insecure Direct Object References (IDOR), and Broken Authentication**.

The vulnerable e-commerce application was created using **Python and Flask**

Things to look up at….

🛡️ How to Teach Mitigation (How to Protect the Site)

Use the checklist below to guide your students through a **code review session** to fix these issues: | Vulnerability Found | Penetration Test Trick | How to Fix / Protect [0.6.14, 0.6.15] | | :--- | :--- | :--- | |

SQL Injection (SQLi) | Enter `admin' --` or `' OR '1'='1` into the login fields to bypass authentication entirely. |

Use Parameterized Queries. Replace the concatenated string query in `/login` with placeholders: `execute("SELECT * FROM users WHERE username=? AND password=?", (username, password))` | |

Plain-Text Passwords | Open the Database or hit the Admin panel to view cleartext records immediately. |

Password Hashing. Use libraries like `werkzeug.security` (`generate_password_hash` and `check_password_hash`) to safely store one-way cryptographic hashes instead of raw passwords [0.6.14]. | |

Reflected XSS | Navigate to `/dashboard?name=<script>alert(document.cookie)</script>` to execute rogue JavaScript. |

Contextual Escaping. Remove the `| safe` filter inside the Jinja dynamic render templates, ensuring everything parses as text strings instead of raw executable HTML code. | |

Insecure Direct Object References (IDOR) | Log in as `bob`, then type `/receipt?order_id=1` into the browser bar to look into `alice`'s purchases. |

@thebrawtalkdigital

Penetration Testing Sandbox Lab

Ownership Verification. When looking up an item, check ownership explicitly: `SELECT * FROM orders WHERE id=? AND user_id=?` matching against the live secure server session token. | |

Broken Access Control | If a user manipulates their session object data or tricks the logic parameters, they get access to `/admin`. |

Strict RBAC (Role-Based Access Control). Always validate the server-side role attributes directly against records in the database, avoiding fuzzy fallback matching like `session.get('username') == 'admin'`. |

Since we are creating it using VS code, make sure you have Visual Studio code if not download it from here (https://code.visualstudio.com/download)

Also confirm if you have python in your machine if not download it here (https://www.python.org/downloads/)

Once you have them installed and .. confirm if python is there by running this command

python –version

Set up your project folder and open it via VS Code ..mine is *vulnerable_shop*

Create the Application File … mine is *app.py* and save it

Install Flask in Vs Code
Open the built-in terminal by going to the top menu and selecting **Terminal > New Terminal** (or press `Ctrl+``). And type the following *pip install Flask*
***Flask** is a lightweight, open-source web framework written in Python used to build web applications and APIs*

Run the Website
In the same terminal window, type the following command to start your mock app..
Python app.py
You'll see output in the terminal saying: **Running on http://127.0.0.1:5000*
Hold Ctrl when on windows and click the link in the terminal or manually open the web browser and navigate *http://127.0.0.1:5000*

After the site is live here are the things to look for in app.py…

1. ***SQL Injection (SQLi) Bypass:***

@thebrawtalkdigital

Penetration Testing Sandbox Lab

- *Go to the **Login** page.*
- *Type `admin' --` into the Username box.*
- *Put anything (like `123`) into the Password box and click Login.*
- ***Result:** It logs straight into the user dashboard as the admin account because the trailing hyphens (`--`) comment out the remaining database query password check!*

2. ***Insecure Direct Object Reference (IDOR):***
   - *Log in using a standard registered test account (or create one using the Register tab).*
   - *Click the link pointing to `/receipt?order_id=1`.*
   - *Manually change the terminal address parameters in the browser URL tracking bar to `order_id=2` or `order_id=3`.*
   - ***Result:** The system will fetch transaction receipt records belonging to other users without validating whether the currently logged-in student actually owns that specific invoice data block.*

Application 2.

Create another application inside our app.py called exploit.py

Let's launch the automation script to test the defense layers of your new mock shop.

Since your web app is running in your current terminal window, we need a separate terminal space to run the attack script so they don't interfere with each other.

1. Open second terminal window
   terminal>new terminal from the top menu bar
   This creates a fresh, clean terminal prompt-terminal #2 while keeping your web server running perfectly in the background on terminal#1

2. Install the Requests Dependency
   The Requests library is a powerful and user-friendly tool for making HTTP requests in Python. It simplifies the process of sending HTTP/1.1 requests, handling responses, and managing various aspects of HTTP communication.

   Use the command…. ***Pip install requests***

3. Launch the exploit using the following command… ***python exploit.py***

@thebrawtalkdigital

Penetration Testing Sandbox Lab

What You Will See in the Output

The script will run instantly and print out the progression logs directly on your screen:

- **SQL Injection Phase:** It will send the payload `admin' --` to `/login`, trick the SQLite query, bypass the authentication block, and confirm a successful breach into the session.
- **IDOR Phase:** It will authenticate normally as `bob`, bypass browser interface restrictions, and harvest and display receipts for Order IDs #1, #2, and so on—exfiltrating data belonging to other accounts.

We placed the application and script in the same directory because:

1. **Easy Execution:** Your terminal path in VS Code is already pointed at `D:\online cyber\vulnerable_shop`. By creating `exploit.py` there, you can just run `python exploit.py` immediately without typing out long file paths.
2. **Project Organization:** It keeps your target application code and your penetration testing attack script grouped together as a complete lab package.

How do we harden the issues…

Open your **`app.py`** file and swap out the vulnerable function blocks with these production-grade secure variations:

Patch 1: Fixing SQL Injection in `/login`

- **The Problem:** The app concatenated untrusted inputs directly into raw SQL strings: `f"SELECT * FROM users WHERE username = '{username}'..."`
- **The Secure Fix:** Use **parameterized SQL statements** (prepared statements). This forces the database engine to treat user input strictly as a literal text value string, completely neutralising structural logic characters like `' --`.

Fix code being …

```
# ❌ OLD VULNERABLE LOGIN LOGIC (Line ~58)
# query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
# user = conn.execute(query).fetchone()
```

@thebrawtalkdigital

Penetration Testing Sandbox Lab

```
#
============================================================
============
# ✅ SECURE PATCHED LOGIN LOGIC (Replace your inside
@app.route('/login') post handler)
#
============================================================
============
query = "SELECT * FROM users WHERE username = ? AND password =
?"
try:
    with get_db() as conn:
        # Pass variables securely inside a safe argument tuple instead of raw
string injections
        user = conn.execute(query, (username, password)).fetchone()
        if user:
            session['user_id'] = user['id']
            session['username'] = user['username']
            session['is_admin'] = user['is_admin']
            return redirect(url_for('dashboard'))
except Exception as e:
    flash(f"Database operation failed safely.")
```

Patch 2: Fixing IDOR in `/receipt`

- **The Problem:** The app only looked up the `order_id` URL parameter without validating if the transaction sequence historically belonged to the currently logged-in account.
- **The Secure Fix:** Enforce an **Authorization Context Integrity check** by linking the query query criteria straight to the server-verified `session['user_id']`.

Fix code is….

```
# ❌ OLD VULNERABLE RECEIPT LOGIC (Line ~115)
# order = conn.execute("SELECT * FROM orders WHERE id = ?",
(oid,)).fetchone()
```

@thebravetalkdigital

Penetration Testing Sandbox Lab

```
#
================================================================
=======
# ✅ SECURE PATCHED RECEIPT LOGIC (Replace your inside
@app.route('/receipt') handler)
#
================================================================
=======
@app.route('/receipt')
def receipt():
    if 'user_id' not in session:
        return "Authentication required.", 401

    oid = request.args.get('order_id')
    current_logged_in_user = session['user_id']

    with get_db() as conn:
        # Force a double relationship lookup verification match constraint
        order = conn.execute(
            "SELECT * FROM orders WHERE id = ? AND user_id = ?",
            (oid, current_logged_in_user)
        ).fetchone()

    if order:
        return f"<h3>System Invoice Output Receipt</h3>Order Reference ID:
{order['id']}<br>Product Item description: {order['item']}<br>Ex-Tax Cost:
${order['price']}"

    # If the order exists but belongs to someone else, it now returns 403 Forbidden
instead of leaking data!
    return "Access Denied: You do not have permission to view this invoice record
data tier.", 403
```

Verifying the Defenses

@thebrawtalkdigital

Penetration Testing Sandbox Lab

1. Save your newly patched `app.py` file.
2. If your Flask terminal screen is running, it will automatically reload the application instance with the new defense layers active.
3. Switch tabs over to your second terminal layout (`Terminal #2`) and run your exploit script again: ***python exploit.py***

Expected Hardened Output Logs

When you run the script this time, the attack automation will break completely:

- **STEP 1** will print `[FAILED] Authentication bypass failed` because the payload `admin' --` is evaluated as a literal username string rather than database command parameters.
- **STEP 2** will throw hard `403 Forbidden` status errors back to the scraper pipeline, shielding user data records from illegal harvesting sweeps.

@thebrawtalkdigital